

Homework Report — Bash Scripting & Operations Automation

Student information

- **Student name:** Nguyễn Tấn Thắng
- **Student ID:** 23127259
- **Class:** 23MMT
- **Submission date:** January 25, 2025

1. Environment and implementation choices

This assignment was implemented and tested on a Linux virtual machine with the following environment:

Item	Value	-----	Virtualization software	UTM	Operating system	Ubuntu Linux	Linux hostname	
web01	Linux user	ubuntu	Web endpoint	python3 -m http.server	Web data			
directory	/home/ubuntu/web01-data	Health URL	http://localhost:8080					
Monitoring configuration	/etc/monitoring.env	Backup configuration						
	/etc/backup.env	Alert method	Local log file	Alert log	/var/log/web01-			
	alerts.log	Health-check cron log	/var/log/web01-health.log	Backup cron log				
	/var/log/web01-backup.log							

A local directory was used as the backup target:

```
/srv/backup-target
```

This represents the fallback destination allowed by the assignment when a second Linux VM is unavailable.

A real SMTP relay was not used. Instead, alert emails were written in email-like format to:

```
/var/log/web01-alerts.log
```

This made all generated alerts observable and suitable for testing and screenshots.

The real configuration files were protected with:

```
sudo chown root:root /etc/monitoring.env /etc/backup.env
sudo chmod 600 /etc/monitoring.env /etc/backup.env
```

Only placeholder `.env.example` files are included in the submitted repository.

2. Submitted project structure

```
web01-ops/
├─ health-check.sh
├─ backup.sh
├─ restore-test.sh
├─ cron/
│   └─ web01-ops.cron
├─ examples/
│   ├── monitoring.env.example
│   └─ backup.env.example
├─ evidence/
│   ├── 01-task1-healthy-run.jpg
│   ├── 02-task1-triggered-alert.jpg
│   ├── 03-task2-backup-success.jpg
│   ├── 04-task2-archive-contents.jpg
│   ├── 05-task2-rotation.jpg
│   ├── 06-task2-restore-test.jpg
│   ├── 07-task2-backup-failure.jpg
│   ├── 08-task3-error-trap.jpg
│   ├── 09-task3-shellcheck.jpg
│   ├── 10-task4-cron-schedule.jpg
│   └─ 11-task4-cron-log.jpg
```

```
|— README.md  
|— REPORT.md  
└— .gitignore
```

All Bash scripts begin with:

```
set -euo pipefail
```

Each script loads its settings from a protected configuration file rather than hard-coding environment-specific values.

Task 1 — Health check with alerting

3. Task 1 implementation

The `health-check.sh` script monitors the Linux host and reports problems only when one or more checks fail.

The script reads the following values from `/etc/monitoring.env`:

```
ALERT_TO="student@example.com"  
DISK_THRESHOLD=85  
RAM_MIN_FREE=15  
SERVICES="ssh cron"  
HEALTH_URL="http://localhost:8080"  
  
MAIL_MODE="log"  
MAIL_LOG="/var/log/web01-alerts.log"
```

The following items are checked:

1. Root filesystem usage. 2. Available RAM percentage. 3. Status of every service listed in `SERVICES`. 4. Availability of the web endpoint.

Problems are appended to an array:

```
ALERTS=()
```

The script does not send a separate message for every failed probe. It completes all checks and sends one combined alert at the end.

A healthy host:

- Produces no script output.
- Sends no alert.
- Exits with status `0`.

An unhealthy host:

- Continues running all probes.
- Combines all findings into one alert.
- Includes the hostname in the alert.
- Exits with a non-zero status.

4. Task 1 acceptance test — Healthy host

Before testing, the following conditions were confirmed:

```
systemctl is-active ssh
systemctl is-active cron
curl -s http://localhost:8080
```

The web endpoint returned:

```
hello from web01
```

The health-check was then executed:

```

sudo /opt/web01-ops/health-check.sh
rc=$?

echo "health-check exit=$rc"
sudo wc -l /var/log/web01-alerts.log

```

Observed result:

```

health-check exit=0
0 /var/log/web01-alerts.log

```

There was no output from the script itself and no alert was written.

Evidence

```

ubuntu@web01:~$ echo "=== TASK 1: HEALTHY RUN ==="
date
hostname

sudo /opt/web01-ops/health-check.sh
rc=$?

echo "health-check exit=$rc"
sudo wc -l /var/log/web01-alerts.log
=== TASK 1: HEALTHY RUN ===
Fri Jul 24 19:15:50 UTC 2026
web01
health-check exit=0
0 /var/log/web01-alerts.log

```

Result

PASS — The healthy host produced no alert and exited with status `0`.

5. Task 1 acceptance test — Triggered alert

To simulate multiple problems:

1. The Python HTTP endpoint was stopped. 2. `DISK_THRESHOLD` was temporarily changed to `1` .

The health-check was executed again:

```
sudo /opt/web01-ops/health-check.sh true
```

The number of alert messages was checked with:

```
sudo grep -c '^Subject:' /var/log/web01-alerts.log
```

Observed result:

```
1
```

The single alert contained multiple findings, including:

```
Root filesystem usage exceeded the configured threshold.  
Web endpoint is unreachable: http://localhost:8080
```

The alert subject contained the hostname:

```
Subject: [web01] HEALTH ALERT
```

Evidence

```

ubuntu@web01:~$ echo "=== TASK 1: TRIGGERED ALERT ==="
date
hostname

sudo /opt/web01-ops/health-check.sh || true

echo
echo "Number of alert messages:"
sudo grep -c '^Subject:' /var/log/web01-alerts.log

echo
echo "Alert content:"
sudo cat /var/log/web01-alerts.log
=== TASK 1: TRIGGERED ALERT ===
Fri Jul 24 19:17:07 UTC 2026
web01

Number of alert messages:
1

Alert content:
--- 2026-07-24T19:17:07+00:00 ---
To: student@example.com
Subject: [web01] HEALTH ALERT

Host: web01
Time: 2026-07-24T19:17:07+00:00

Problems detected:
- Root filesystem usage is 48% (threshold: 1%)
- Web endpoint is unreachable: http://localhost:8080

```

Result

PASS — Exactly one combined alert was produced, and all configured probes were still evaluated.

After testing, the disk threshold was restored to `85`, and the web endpoint was restarted.

Task 2 — Automated backup with safe cleanup

6. Task 2 implementation

The `backup.sh` script performs the following operations:

1. Loads variables from `/etc/backup.env`.
2. Validates all required variables.
3. Creates a temporary working directory.
4. Registers an `EXIT` cleanup handler.
5. Generates an MD5

manifest. 6. Creates a compressed timestamped archive. 7. Excludes `.log` and `.tmp` files. 8. Copies the archive to the backup destination. 9. Rotates expired archives. 10. Sends a success report. 11. Sends a failure report if any step fails. 12. Removes its temporary working directory in all cases.

The configuration used was:

```
DATA_DIR="/home/ubuntu/web01-data"
ALERT_TO="student@example.com"
DEST="/srv/backup-target"
RETAIN_DAYS=7

MAIL_MODE="log"
MAIL_LOG="/var/log/web01-alerts.log"

FORCE_FAILURE=false
```

Required variables are guarded using parameter validation such as:

```
: "${DATA_DIR:?data dir required}"
```

The temporary directory is always cleaned using:

```
trap cleanup EXIT
```

If the script exits with a non-zero status, the cleanup function generates a `BACKUP FAILED` alert before removing temporary data.

7. Task 2 acceptance test — Successful backup

The backup script was run with:


```
sudo /opt/web01-ops/backup.sh
```

The destination was inspected using:

```
sudo ls -lh /srv/backup-target
```

A timestamped archive was created successfully:

```
web01-backup-web01-YYYYMMDD-HHMMSS.tar.gz
```

The success report contained:

```
Subject: [web01] Backup OK
```

It also included:

- Archive name.
- Archive size.
- Destination path.
- Hostname.

Evidence

```

ubuntu@web01:~$ echo "=== TASK 2: SUCCESSFUL BACKUP ==="
date
hostname

sudo /opt/web01-ops/backup.sh

echo
echo "Destination archives:"
sudo ls -lh /srv/backup-target

echo
echo "Backup report:"
sudo cat /var/log/web01-alerts.log
=== TASK 2: SUCCESSFUL BACKUP ===
Fri Jul 24 19:18:38 UTC 2026
web01
Backup OK: web01-backup-web01-20260724-191838.tar.gz (4.0K) -> /srv/backup-target

Destination archives:
total 4.0K
-rw-r--r-- 1 root root 340 Jul 24 19:18 web01-backup-web01-20260724-191838.tar.gz

Backup report:
--- 2026-07-24T19:18:38+00:00 ---
To: student@example.com
Subject: [web01] Backup OK

Host: web01
Time: 2026-07-24T19:18:38+00:00
Archive: web01-backup-web01-20260724-191838.tar.gz
Size: 4.0K
Destination: /srv/backup-target

```

Result

PASS — A valid timestamped `.tar.gz` archive was created at the configured destination.

8. Task 2 acceptance test — Archive contents and exclusions

The contents of the latest archive were inspected with:

```
sudo tar -tzf "$LATEST"
```

The archive contained the manifest and important data files, including:

```
manifest.txt
data/index.html
```

```
data/sub/info.txt
```

The source directory also contained:

```
debug.log
cache.tmp
```

These files were intentionally created to test the exclusion rules.

The archive was checked using:

```
if sudo tar -tzf "$LATEST"
  grep -Eq '\.(logtmp)$'; then
    echo "FAILED: .log or .tmp exists in archive"
else
    echo "PASSED: no .log or .tmp in archive"
fi
```

Observed result:

```
PASSED: no .log or .tmp in archive
```

Evidence

```
ubuntu@web01:~$ echo "=== TASK 2: ARCHIVE CONTENTS ==="
sudo tar -tzf "$LATEST"
echo
if sudo tar -tzf "$LATEST" |
  grep -Eq '\.(log|tmp)$'; then
    echo "FAILED: .log or .tmp exists in archive"
else
    echo "PASSED: no .log or .tmp in archive"
fi
=== TASK 2: ARCHIVE CONTENTS ===
manifest.txt
data/
data/index.html
data/sub/
data/sub/info.txt
PASSED: no .log or .tmp in archive
```

Result

PASS — The archive included the manifest and important files while excluding `.log` and `.tmp` files.

9. Task 2 acceptance test — Archive rotation

To verify archive rotation, a copy of an existing archive was created and assigned a modification time of ten days ago:

```
sudo cp "$LATEST" \  
/srv/backup-target/web01-backup-old-test.tar.gz  
  
sudo touch -d "10 days ago" \  
/srv/backup-target/web01-backup-old-test.tar.gz
```

Because `RETAIN_DAYS=7`, the old test archive should be removed during the next backup.

The backup script was run again:

```
sudo /opt/web01-ops/backup.sh
```

The destination was inspected before and after the operation.

Observed result:

```
PASSED: old archive was deleted
```

Evidence

```

ubuntu@web01:~$ echo "=== BEFORE ROTATION ==="
sudo ls -lh --time-style=long-iso /srv/backup-target

echo
sudo /opt/web01-ops/backup.sh

echo
echo "=== AFTER ROTATION ==="
sudo ls -lh --time-style=long-iso /srv/backup-target

echo
if sudo test \
! -f /srv/backup-target/web01-backup-old-test.tar.gz; then
    echo "PASSED: old archive was deleted"
else
    echo "FAILED: old archive still exists"
fi
=== BEFORE ROTATION ===
total 8.0K
-rw-r--r-- 1 root root 340 2026-07-14 19:19 web01-backup-old-test.tar.gz
-rw-r--r-- 1 root root 340 2026-07-24 19:18 web01-backup-web01-20260724-191838.tar.gz

Backup OK: web01-backup-web01-20260724-192009.tar.gz (4.0K) -> /srv/backup-target

=== AFTER ROTATION ===
total 8.0K
-rw-r--r-- 1 root root 340 2026-07-24 19:18 web01-backup-web01-20260724-191838.tar.gz
-rw-r--r-- 1 root root 340 2026-07-24 19:20 web01-backup-web01-20260724-192009.tar.gz

PASSED: old archive was deleted

```

Result

PASS — Archives older than the configured retention period were deleted.

10. Task 2 acceptance test — Verified restore

The `restore-test.sh` script performs a real restore test rather than only checking whether an archive exists.

It:

1. Selects the latest backup archive.
2. Extracts it into a temporary directory.
3. Locates the included manifest.
4. Runs `md5sum -c` against the restored files.
5. Reports the number of matched files.
6. Removes the temporary restore directory.

The restore test was executed with:

```

sudo /opt/web01-ops/restore-test.sh
rc=$?

```

```
echo "restore-test exit=$rc"
```

Observed output:

```
./index.html: OK
./sub/info.txt: OK
Restore test PASSED: 2/2 files matched
restore-test exit=0
```

The exact number of files depends on the contents of the source data directory. In this test, every file recorded in the manifest matched its expected checksum.

Evidence

```
ubuntu@web01:~$ echo "=== TASK 2: VERIFIED RESTORE ==="
date
hostname

sudo /opt/web01-ops/restore-test.sh
rc=$?

echo "restore-test exit=$rc"
=== TASK 2: VERIFIED RESTORE ===
Fri Jul 24 19:20:39 UTC 2026
web01
./index.html: OK
./sub/info.txt: OK
Restore test PASSED: 2/2 files matched for web01-backup-web01-20260724-192009.tar.gz
restore-test exit=0
```

Result

PASS — The latest backup was successfully extracted and every restored file matched the stored manifest.

11. Task 2 acceptance test — Failure handling and cleanup

A separate test configuration was created with an invalid data directory:

```
DATA_DIR="/no/such/path"
```

The backup script was executed using that temporary configuration:

```
sudo env BACKUP_CONFIG=/tmp/backup-fail.env \  
/opt/web01-ops/backup.sh true
```

The test verified two requirements:

1. A `BACKUP FAILED` alert was generated. 2. The temporary working directory was removed.

The cleanup check returned:

```
PASSED: temporary directory cleaned
```

The alert log contained exactly one failure message:

```
Subject: [web01] BACKUP FAILED
```

Evidence

```

ubuntu@web01:~$ echo "=== TASK 2: BACKUP FAILURE ==="
date
hostname

if sudo find /tmp \
  -maxdepth 1 \
  -type d \
  -name 'web01-backup.*' |
  grep -q .; then
  echo "FAILED: temporary directory remains"
else
  echo "PASSED: temporary directory cleaned"
fi

echo
echo "Number of failure alerts:"
sudo grep -c '^Subject:.*BACKUP FAILED' \
  /var/log/web01-alerts.log

echo
sudo cat /var/log/web01-alerts.log
=== TASK 2: BACKUP FAILURE ===
Fri Jul 24 19:21:31 UTC 2026
web01
PASSED: temporary directory cleaned

Number of failure alerts:
1

--- 2026-07-24T19:21:23+00:00 ---
To: student@example.com
Subject: [web01] BACKUP FAILED

Host: web01
Time: 2026-07-24T19:21:23+00:00
Exit code: 1
Line: 102
Command: false

```

Result

PASS — The failure was reported, and no temporary backup directory remained.

Task 3 — Error trapping and linting

12. ERR trap implementation

The backup script contains an error handler connected to the Bash `ERR` trap:

```
trap 'on_error $LINENO "$BASH_COMMAND"' ERR
```


The handler captures:

- Hostname.
- Time.
- Failing line number.
- Exact failing command.
- Exit code.

This information is included in the generated failure report.

A controlled fault is enabled through the test-only configuration variable:

```
FORCE_FAILURE=true
```

When enabled, the script attempts to execute a real failing command similar to:

```
tar -czf "$WORK_DIR/forced-error.tgz" /no/such/dir
```

A marker command exists after the failing command:

```
THIS LINE MUST NOT RUN
```

This marker is used to verify that execution stops at the failing line.

13. Task 3 acceptance test — Forced error

The forced-error test was executed with:

```
sudo env BACKUP_CONFIG=/tmp/backup-force-error.env \  
/opt/web01-ops/backup.sh \  
> /tmp/forced-error-output.txt 2>&1 true
```

The output was checked to ensure the marker after the failing command was absent.

Observed result:

```
PASSED: code after the error did not run
```

The generated error report contained:

```
Host: web01  
Exit code: 2  
Line: 128  
Command: tar -czf "$WORK_DIR/forced-error.tgz" /no/such/dir
```

The exact line number may change if the script is edited, but the report correctly identifies the actual failing line.

Exactly one `BACKUP FAILED` alert was generated.

Evidence

```

ubuntu@web01:~$ echo "=== TASK 3: ERR TRAP ==="
date
hostname

cat /tmp/forced-error-output.txt

echo
if grep -q \
  "THIS LINE MUST NOT RUN" \
  /tmp/forced-error-output.txt; then
  echo "FAILED: code after the error was executed"
else
  echo "PASSED: code after the error did not run"
fi

echo
echo "Number of BACKUP FAILED alerts:"
sudo grep -c '^Subject:.*BACKUP FAILED' \
  /var/log/web01-alerts.log

echo
sudo cat /var/log/web01-alerts.log
=== TASK 3: ERR TRAP ===
Fri Jul 24 19:22:41 UTC 2026
web01
tar: Removing leading '/' from member names
tar: /no/such/dir: Cannot stat: No such file or directory
tar: Exiting with failure status due to previous errors

PASSED: code after the error did not run

Number of BACKUP FAILED alerts:
1

--- 2026-07-24T19:22:34+00:00 ---
To: student@example.com
Subject: [web01] BACKUP FAILED

Host: web01
Time: 2026-07-24T19:22:34+00:00
Exit code: 2
Line: 128
Command: tar -czf "$WORK_DIR/forced-error.tgz" /no/such/dir

```

Result

PASS — The real fault stopped execution at the correct command, and the error report included the line number, command, exit code, and hostname.

14. ShellCheck before correction

During the first ShellCheck run, malformed inline ShellCheck directives were detected.

The incorrect directive format was similar to:



ShellCheck reported parsing errors including:

```
SC1072
SC1073
```

The directive was corrected to:



Evidence before correction

```
ubuntu@web01:/opt/web01-ops$ echo "=== TASK 3: SHELLCHECK ==="
date
hostname

shellcheck ./*.sh
rc=$?

echo "shellcheck exit=$rc"
=== TASK 3: SHELLCHECK ===
Fri Jul 24 19:27:09 UTC 2026
web01
shellcheck exit=0
```

15. ShellCheck final result

ShellCheck was run on every submitted script:

```
cd /opt/web01-ops

shellcheck ./*.sh
rc=$?

echo "shellcheck exit=$rc"
```

Final result:

```
shellcheck exit=0
```

No warnings or errors were printed.

Evidence after correction

```
ubuntu@web01:/opt/web01-ops$ echo "=== TASK 3: SHELLCHECK ==="
date
hostname

shellcheck ./*.sh
rc=$?

echo "shellcheck exit=$rc"
=== TASK 3: SHELLCHECK ===
Fri Jul 24 19:27:09 UTC 2026
web01
shellcheck exit=0
```

Result

PASS — All submitted Bash scripts pass ShellCheck with exit code 0 .

Task 4 — Scheduling with cron

16. Cron configuration

The project includes:

```
cron/web01-ops.cron
```

It was installed as:

```
/etc/cron.d/web01-ops
```

The cron file sets an explicit `PATH` because cron jobs run with a minimal environment.

The required production schedule is:

```
*/5 * * * * root /opt/web01-ops/health-check.sh >> /var/log/web01-h  
0 2 * * * root /opt/web01-ops/backup.sh >> /var/log/web01-backup.log
```

This means:

- `health-check.sh` runs every five minutes.
- `backup.sh` runs every day at 02:00.
- Output is redirected to files under `/var/log`.

The cron service was confirmed active with:

```
systemctl is-active cron
```

Observed result:

```
active
```

17. Task 4 acceptance test — Installed schedule

The installed cron configuration was displayed using:

```
sudo cat /etc/cron.d/web01-ops
```

The output showed both required jobs:

```
*/5 * * * *
0 2 * * *
```

Evidence

```
ubuntu@web01:/opt/web01-ops$ echo "=== TASK 4: INSTALLED CRON SCHEDULE ==="
date
hostname

systemctl is-active cron

echo
sudo cat /etc/cron.d/web01-ops
=== TASK 4: INSTALLED CRON SCHEDULE ===
Fri Jul 24 19:28:30 UTC 2026
web01
active

SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Install this project at /opt/web01-ops before using this cron.d file.
# Health check every 5 minutes.
*/5 * * * * root /bin/bash -c 'echo "$(/bin/date -Is) health-check start"; /opt/web01-ops/health-check.sh; rc=$?; echo "$(/bin/date -Is) health-check
exit=$rc"; exit 0' >> /var/log/web01-health.log 2>&1

# Backup every day at 02:00.
0 2 * * * root /bin/bash -c 'echo "$(/bin/date -Is) backup start"; /opt/web01-ops/backup.sh; rc=$?; echo "$(/bin/date -Is) backup exit=$rc"; exit 0'
>> /var/log/web01-backup.log 2>&1

# Grading demo only: temporarily replace */5 above with */1, then restore */5.
```

Result

PASS — Both scripts are scheduled with the required production times.

18. Task 4 acceptance test — One-minute demonstration

For demonstration, the health-check schedule was temporarily changed from:

```
*/5 * * * *
```

to:

```
*/1 * * * *
```

Before the test:

1. The health log was cleared. 2. The alert log was cleared. 3. The web endpoint was stopped.

Cron then automatically executed the health-check on successive minute boundaries.

The health log showed multiple scheduled executions with different timestamps:

```
health-check start  
health-check exit=1  
health-check start  
health-check exit=1
```

The alert log also showed that the scheduled health-check detected the stopped web endpoint.

After collecting the evidence, the production schedule was restored to:

```
*/5 * * * *
```

The Python web endpoint was also restarted.

Evidence


```

ubuntu@web01:/opt/web01-ops$ echo "=== TASK 4: ONE-MINUTE CRON DEMO ==="
date
hostname

echo
echo "Temporary cron schedule:"
sudo grep 'health-check.sh' /etc/cron.d/web01-ops

echo
echo "Cron execution log:"
sudo grep -E 'health-check (start|exit)' \
  /var/log/web01-health.log |
  tail -n 8

echo
echo "Alert generated by cron:"
sudo tail -n 16 /var/log/web01-alerts.log
=== TASK 4: ONE-MINUTE CRON DEMO ===
Fri Jul 24 19:31:27 UTC 2026
web01

Temporary cron schedule:
*/1 * * * root /bin/bash -c 'echo "$(/bin/date -Is) health-check start"; /opt/web01-ops/health-check.sh; rc=$?; echo "$(/bin/date -Is) health-check
exit=$rc"; exit 0' >> /var/log/web01-health.log 2>&1

Cron execution log:
2026-07-24T19:30:01+00:00 health-check start
2026-07-24T19:30:01+00:00 health-check exit=1
2026-07-24T19:31:01+00:00 health-check start
2026-07-24T19:31:01+00:00 health-check exit=1

Alert generated by cron:
Subject: [web01] HEALTH ALERT

Host: web01
Time: 2026-07-24T19:30:01+00:00

Problems detected:
- Web endpoint is unreachable: http://localhost:8080
--- 2026-07-24T19:31:01+00:00 ---
To: student@example.com
Subject: [web01] HEALTH ALERT

Host: web01
Time: 2026-07-24T19:31:01+00:00

```

Result

PASS — The health-check ran automatically under cron and detected the triggered failure.

19. Why a script may work manually but fail under cron

A script that works in an interactive terminal may fail under cron because cron provides a much smaller execution environment.

Common differences include:

1. Minimal `PATH`

Commands such as `curl`, `tar`, `find`, `rsync`, or `python3` may not be found if the cron environment does not include their directories.

2. Different working directory

A cron job does not automatically start in the project directory. Relative paths may therefore point to the wrong location.

3. Different user

A job may run as `root` or another configured user rather than the interactive user.

4. Different `HOME` variable

A path such as:

```
~/web01-data
```

may resolve to `/root/web01-data` when run as root.

5. Missing interactive shell configuration

Files such as `.bashrc` and `.profile` are generally not loaded for cron jobs.

6. Permissions

A cron user may not have permission to read configuration files, write logs, access source data, or write to the destination.

7. No interactive terminal

A cron job cannot answer prompts or display output to a user.

These problems were avoided by:

- Using absolute paths.
- Setting `PATH` in the cron file.
- Using absolute configuration paths.
- Redirecting output to log files.
- Configuring permissions explicitly.
- Avoiding interactive commands.

20. Acceptance-criteria summary

Requirement Evidence Result ----- Healthy health-check is silent 01-task1-healthy-run.jpg PASS Healthy health-check exits 0 01-task1-healthy-run.jpg PASS Triggered condition sends one alert 02-task1-triggered-alert.jpg PASS All health probes still run 02-task1-triggered-alert.jpg PASS Timestamped archive created 03-task2-backup-success.jpg PASS Backup success report generated 03-task2-backup-success.jpg PASS .log and .tmp excluded 04-task2-archive-contents.jpg PASS Old archives rotated 05-task2-rotation.jpg PASS Latest backup restores successfully 06-task2-restore-test.jpg PASS Manifest checks all pass 06-task2-restore-test.jpg PASS Backup failure sends alert 07-task2-backup-failure.jpg PASS Temporary directory cleaned 07-task2-backup-failure.jpg PASS Forced error stops execution 08-task3-error-trap.jpg PASS Error alert includes required details 08-task3-error-trap.jpg PASS ShellCheck warnings corrected 09-task3-shellcheck.jpg PASS Final ShellCheck output is clean 09-task3-shellcheck.jpg PASS Health-check scheduled every 5 minutes 10-task4-cron-schedule.jpg PASS Backup scheduled daily at 02:00 10-task4-cron-schedule.jpg PASS Cron automatically runs health-check 11-task4-cron-log.jpg PASS Triggered cron run produces an alert 11-task4-cron-log.jpg PASS

21. Safety and reliability notes

The implementation uses several measures to avoid misleading or unsafe backup results.

Strict mode

Every script uses:

```
set -euo pipefail
```

This helps detect:

- Failed commands.
- Unset variables.
- Failures inside pipelines.

Temporary working directory

Backup preparation is performed in a temporary directory rather than directly at the destination.

Cleanup trap

The temporary working directory is removed regardless of success or failure.

Failure alerting

A non-zero backup result generates a `BACKUP FAILED` message.

Verified restore

A backup is not treated as fully proven until its files are restored and checked against the included manifest.

Protected configuration

Real configuration files are stored in `/etc`, owned by root, and assigned permission mode `600`.

Secret handling

The real `.env` files are not committed to Git. Only `.env.example` files with placeholder values are included.

22. Git repository history

The project was maintained in a Git repository with separate commits for major implementation stages.

Repository: <https://github.com/thangak18/web01-ops>

Commit history:

```
e76e973 Add acceptance evidence and final report
752e660 Add cron automation
5d2826b Add gitignore for sensitive configs
ab5e6a0 Add restore verification
c003751 Add verified rotating backup
4ecbea5 Implement host health monitoring
```

23. Final conclusion

The completed operations toolkit allows the `web01` Linux host to monitor, back up, verify, rotate, and report its own operational state without requiring manual execution.

The final implementation provides:

- Silent health monitoring when the host is healthy.
- One combined alert when problems are detected.
- Timestamped compressed backups.
- Exclusion of unnecessary log and temporary files.
- A checksum manifest for every backup.
- A real verified restore process.
- Automatic cleanup after failures.
- Detailed Bash error trapping.
- Clean ShellCheck output.
- Automatic execution through cron.
- Log-based evidence for scheduled jobs.

All required acceptance scenarios were executed on the Ubuntu VM and documented with labelled evidence.